



**Tribhuvan University**  
**Faculty of Humanities and Social Sciences**

**A Lab Report On**  
**Advance Operating System (MCA502)**

**Submitted to**  
**Department of Humanities and Social Science**  
**Patan Multiple Campus**

*In partial fulfillment of the requirements for the Master in Computer Application*

**Submitted by**  
**Bishal Rijal**  
**MCA 1<sup>st</sup> Semester**  
**Tu Register no: 6-2-1133-50-2019**

**Submitted to**  
**Asst. Prof. Bhupendra Ram Luhar**

---

## Table of Contents

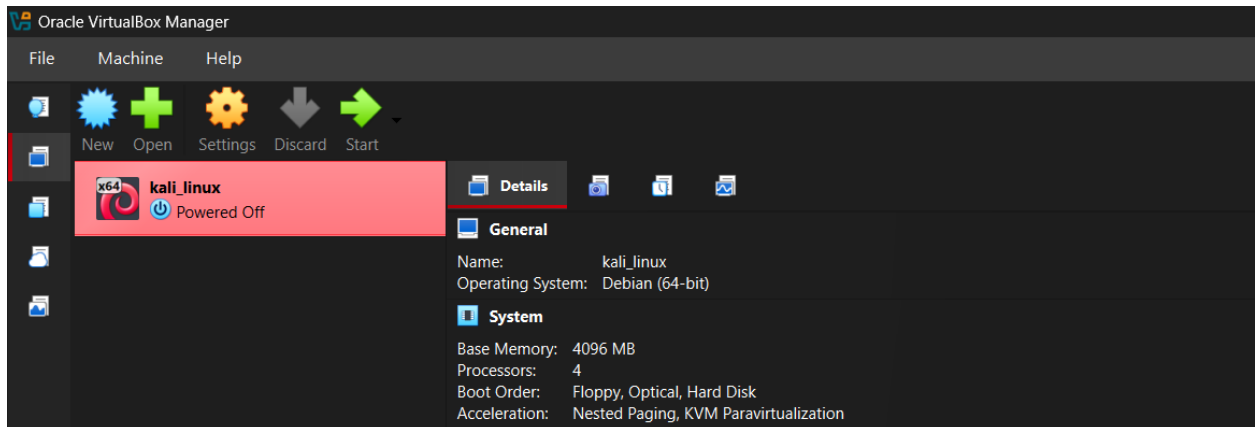
|                                                                                                                                 |    |
|---------------------------------------------------------------------------------------------------------------------------------|----|
| Lab-1: Installation of Linux in Virtual machine and using some Linux command.....                                               | 1  |
| a. Basic command (pwd, ls, cd, cd., mkdir, rmdir, rm – r, cp, mv, locate etc.).....                                             | 1  |
| b. Intermediate command (echo, cat, sudo, sudo password, su, sudo bash, exit etc.) .....                                        | 3  |
| c. Process creation in Linux using fork() System call. ....                                                                     | 4  |
| Lab-2: Write a program to simulate producer – consumer problem using semaphore.....                                             | 5  |
| Lab-3: write a program to simulate following CPU scheduling algorithms to find average turnaround time<br>and waiting time..... | 7  |
| Lab-4: Write program to simulate the following contiguous memory allocation algorithm.....                                      | 10 |
| Lab-5: Write a program to simulate following page replacement algorithm and find the page fault.....                            | 13 |
| Lab-6: Write a Program to simulate following disk scheduling algorithms .....                                                   | 15 |

## Lab-1: Installation of Linux in Virtual machine and using some Linux command

### Theory:

A Virtual Machine (VM) is a software-based emulation of a physical computer that runs an operating system (the "guest") on top of another operating system (the "host"), sharing the host's hardware resources like CPU, memory, and storage. Software called a hypervisor (e.g., VirtualBox, VMware) creates and manages these VMs, allowing multiple isolated operating systems to run on a single physical machine.

Here I have downloaded the kali linux Operating system and installed it on virtual machine.



### a. Basic command (pwd, ls, cd, cd.., mkdir, rmdir, rm – r, cp, mv, locate etc.)

**Pwd:** Print the absolute path of the current working directory.

```
bishal@kali: ~  
Session Actions Edit View Help  
(bishal@kali)-[~]  
$ pwd  
/home/bishal  
(bishal@kali)-[~]  
$
```

**Ls:** Lists the files and sub – directories present in a directory.

```
(bishal@kali)-[~]  
$ ls  
Desktop    Downloads  Pictures   Public     Videos  
Documents  Music      Projects   Templates  
(bishal@kali)-[~]  
$
```

**Cd:** Change the current working directory to the specified path.

```
(bishal@kali)-[~]  
$ cd Documents  
(bishal@kali)-[~/Documents]  
$
```

**Mkdir:** Creates a new directory.

```
(bishal@kali)-[~]
$ mkdir mca

(bishal@kali)-[~]
$ ls
Desktop  Downloads  Music  Projects  Templates
Documents mca        Pictures Public    Videos
```

**Rmdir:** Removes an empty directory.

```
(bishal@kali)-[~]
$ rmdir mca

(bishal@kali)-[~]
$ ls
Desktop  Downloads  Pictures  Public  Videos
Documents Music      Projects  Templates
```

**Cp:** Copies of files or directories from a source to a destination.

```
(bishal@kali)-[~]
$ touch bishal.txt

(bishal@kali)-[~]
$ ls
bishal.txt  Documents  Music  Projects  Templates
Desktop    Downloads  Pictures Public    Videos

(bishal@kali)-[~]
$ cp bishal.txt rijal.txt

(bishal@kali)-[~]
$ ls
bishal.txt  Documents  Music  Projects  rijal.txt  Videos
Desktop    Downloads  Pictures Public    Templates
```

**Mv:** Moves or renames files and directories.

```
(bishal@kali)-[~]
$ mv rijal.txt brijal.txt

(bishal@kali)-[~]
$ ls
bishal.txt  Desktop  Downloads  Pictures  Public  Videos
brijal.txt  Documents Music      Projects  Templates
```

**Locate:** Searches for files and directories by name using a pre-built file-name database.

```
(bishal@kali)-[~]  
$ locate brijal.txt  
/var/lib/locate/locate.db: No such file or directory
```

**b. Intermediate command (echo, cat, sudo, sudo password, su, sudo bash, exit etc.)**

**Echo:** Prints text to the terminal.

```
(bishal@kali)-[~]  
$ echo "Hello from Bishal Rijal"  
Hello from Bishal Rijal
```

**Cat:** Displays the contents of a file.

```
(bishal@kali)-[~]  
$ cat bishal.txt  
Bishal Rijal is a student of Patan Multiple Campus.
```

**Sudo:** Executes a single command with superuser (administrator / root) privileges.

```
(bishal@kali)-[~]  
$ sudo apt update  
[sudo] password for bishal:  
Hit:1 http://http.kali.org/kali kali-rolling InRelease  
1343 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

**sudo passwd:** Sets or changes the password of the root user account.

```
(bishal@kali)-[~]  
$ sudo passwd  
New password:  
Retype new password:  
passwd: password updated successfully
```

**Su:** Switches the current session to another user account, commonly the root user.

```
(bishal@kali)-[~]  
$ su  
Password:  
(root@kali)-[/home/bishal]  
# su bishal
```

**Sudo bash:** Opens a new Bash shell that runs with root privileges.

```
(bishal@kali)-[~]  
$ sudo bash  
(root@kali)-[/home/bishal]  
#
```

**Exit:** Terminates (exits) the current shell session, returning to the previous shell or user.

```
(root@kali)-[/home/bishal]
# exit
exit
```

### c. Process creation in Linux using fork() System call.

#### Theory:

fork() is a system call in Linux/Unix used to create a new process by duplicating the calling process. The original process is called the parent, and the newly created copy is called the child. After the call, both processes run independently from the same point in the code, each with its own memory and PID.

#### Source Code:

```
bishal@kali: ~
Session Actions Edit View Help
GNU nano 9.0 fork.c
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>

int main(){
pid_t pid;
pid=fork()

if(pid<0){
fprintf(stderr, "Fork Failed\n")
return 1;
}else if(pid==0){
printf("Helo from the child preocess!!\n");
printf("Child PID: %d, parent pid: %d\n\n", getpid(), getppid());
}else{
printf("Hello from the parent processing.");
printf("parent pid: %d, newly created child pid: %d\n\n", getpid(), pid);
}
return 0;
}
```

#### Output:

```
bishal@kali: ~
Session Actions Edit View Help
(bishal@kali)-[~]
$ nano fork.c
(bishal@kali)-[~]
$ nano fork.c
(bishal@kali)-[~]
$ nano fork.c
(bishal@kali)-[~]
$ gcc fork.c -o fork
Hello from the PARENT process!
Parent PID: 31335, Newly Created Child PID: 31336
Hello from the CHILD process!
Child PID: 31336, Parent PID: 1
```

## Lab-2: Write a program to simulate producer – consumer problem using semaphore.

### Source Code:

```
import threading
import time
import random

BUFFER_SIZE = 5
buffer = []

empty = threading.Semaphore(BUFFER_SIZE)
full = threading.Semaphore(0)
mutex = threading.Semaphore(1)

def producer():
    for i in range(4):
        item = random.randint(1, 100)
        empty.acquire()
        mutex.acquire()
        buffer.append(item)
        print(f'Producer produced: {item}')
        print("Buffer:", buffer)
        mutex.release()
        full.release()
        time.sleep(1)

def consumer():
    for i in range(4):
        full.acquire()
        mutex.acquire()
        item = buffer.pop(0)
        print(f'Consumer consumed: {item}')
        print("Buffer:", buffer)
        mutex.release()
        empty.release()
        time.sleep(1)

p = threading.Thread(target=producer)
c = threading.Thread(target=consumer)

p.start()
c.start()

p.join()
c.join()
print("Producer-Consumer simulation completed.")
```

**Output:**

```
Producer produced: 91
Buffer: [91]
Consumer consumed: 91
Buffer: []
Producer produced: 63
Buffer: [63]
Consumer consumed: 63
Buffer: []
Producer produced: 66
Buffer: [66]
Consumer consumed: 66
Buffer: []
Producer produced: 64
Buffer: [64]
Consumer consumed: 64
Buffer: []
Producer-Consumer simulation completed.
```



**Lab-3: write a program to simulate following CPU scheduling algorithms to find average turnaround time and waiting time.**

- a. FCFS**
- b. SJF**
- c. Priority scheduling**
- d. Round Robin**

**Source Code:**

```
def fcfs(processes):
    wt = [0] * len(processes)
    tat = [0] * len(processes)
    for i in range(1, len(processes)):
        wt[i] = wt[i - 1] + processes[i - 1][1]
    for i in range(len(processes)):
        tat[i] = wt[i] + processes[i][1]
    print("\nFCFS Scheduling")
    print("PID\tBT\tWT\tTAT")
    for i, p in enumerate(processes):
        print(f'{p[0]}\t{p[1]}\t{wt[i]}\t{tat[i]}')
    print("Average Waiting Time =", sum(wt) / len(processes))
    print("Average Turnaround Time =", sum(tat) / len(processes))
```

```
def sjf(processes):
    processes = sorted(processes, key=lambda x: x[1])
    wt = [0] * len(processes)
    tat = [0] * len(processes)
    for i in range(1, len(processes)):
        wt[i] = wt[i - 1] + processes[i - 1][1]
    for i in range(len(processes)):
        tat[i] = wt[i] + processes[i][1]
    print("\nSJF Scheduling")
    print("PID\tBT\tWT\tTAT")
    for i, p in enumerate(processes):
        print(f'{p[0]}\t{p[1]}\t{wt[i]}\t{tat[i]}')
    print("Average Waiting Time =", sum(wt) / len(processes))
    print("Average Turnaround Time =", sum(tat) / len(processes))
```

```
def priority(processes):
    processes = sorted(processes, key=lambda x: x[2])
    wt = [0] * len(processes)
    tat = [0] * len(processes)
    for i in range(1, len(processes)):
        wt[i] = wt[i - 1] + processes[i - 1][1]
    for i in range(len(processes)):
        tat[i] = wt[i] + processes[i][1]
    print("\nPriority Scheduling")
    print("PID\tBT\tPriority\tWT\tTAT")
```

```

for i, p in enumerate(processes):
    print(f' {p[0]} \t {p[1]} \t {p[2]} \t {wt[i]} \t {tat[i]} ')
print("Average Waiting Time =", sum(wt) / len(processes))
print("Average Turnaround Time =", sum(tat) / len(processes))

```

```

def round_robin(processes, quantum):
    n = len(processes)
    remaining = [p[1] for p in processes]
    wt = [0] * n
    tat = [0] * n
    time = 0
    while True:
        done = True
        for i in range(n):
            if remaining[i] > 0:
                done = False
                if remaining[i] > quantum:
                    time += quantum
                    remaining[i] -= quantum
                else:
                    time += remaining[i]
                    wt[i] = time - processes[i][1]
                    remaining[i] = 0
            if done:
                break
        for i in range(n):
            tat[i] = processes[i][1] + wt[i]
        print("\nRound Robin Scheduling")
        print("PID\tBT\tWT\tTAT")
        for i, p in enumerate(processes):
            print(f' {p[0]} \t {p[1]} \t {wt[i]} \t {tat[i]} ')
        print("Average Waiting Time =", sum(wt) / n)
        print("Average Turnaround Time =", sum(tat) / n)

```

```

processes = [
    ("P1", 6, 2),
    ("P2", 8, 1),
    ("P3", 7, 3),
    ("P4", 3, 2)
]

```

```

fcfs(processes)
sjf(processes)
priority(processes)
round_robin(processes, 2)

```

## Output:

### FCFS Scheduling

| PID | BT | WT | TAT |
|-----|----|----|-----|
| P1  | 6  | 0  | 6   |
| P2  | 8  | 6  | 14  |
| P3  | 7  | 14 | 21  |
| P4  | 3  | 21 | 24  |

Average Waiting Time = 10.25

Average Turnaround Time = 16.25

### SJF Scheduling

| PID | BT | WT | TAT |
|-----|----|----|-----|
| P4  | 3  | 0  | 3   |
| P1  | 6  | 3  | 9   |
| P3  | 7  | 9  | 16  |
| P2  | 8  | 16 | 24  |

Average Waiting Time = 7.0

Average Turnaround Time = 13.0

### Priority Scheduling

| PID | BT | Priority | WT | TAT |
|-----|----|----------|----|-----|
| P2  | 8  | 1        | 0  | 8   |
| P1  | 6  | 2        | 8  | 14  |
| P4  | 3  | 2        | 14 | 17  |
| P3  | 7  | 3        | 17 | 24  |

Average Waiting Time = 9.75

Average Turnaround Time = 15.75

### Round Robin Scheduling

| PID | BT | WT | TAT |
|-----|----|----|-----|
| P1  | 6  | 11 | 17  |
| P2  | 8  | 15 | 23  |
| P3  | 7  | 17 | 24  |
| P4  | 3  | 12 | 15  |

Average Waiting Time = 13.75

Average Turnaround Time = 19.75

## Lab-4: Write program to simulate the following contiguous memory allocation algorithm

### a. First Fit

#### Source Code:

```
# First Fit Memory Allocation
def first_fit(blocks, processes):
    allocation = [-1] * len(processes)

    for i in range(len(processes)):
        for j in range(len(blocks)):
            if blocks[j] >= processes[i]:
                allocation[i] = j
                blocks[j] -= processes[i]
                break

    print("\nProcess\tSize\tBlock")
    for i in range(len(processes)):
        if allocation[i] != -1:
            print(f"P{i+1}\t\t{processes[i]}\tB{allocation[i]+1}")
        else:
            print(f"P{i+1}\t\t{processes[i]}\tNot Allocated")

blocks = list(map(int, input("Enter memory block sizes: ").split()))
processes = list(map(int, input("Enter process sizes: ").split()))

first_fit(blocks, processes)
```

#### Output:

```
Enter memory block sizes: 100 500 200 300 600
Enter process sizes: 212 417 112 426
```

| Process | Size | Block         |
|---------|------|---------------|
| P1      | 212  | B2            |
| P2      | 417  | B5            |
| P3      | 112  | B2            |
| P4      | 426  | Not Allocated |

## b. Best Fit

### Source Code:

```
def best_fit(blocks, processes):
    allocation = [-1] * len(processes)

    for i in range(len(processes)):
        best = -1

        for j in range(len(blocks)):
            if blocks[j] >= processes[i]:
                if best == -1 or blocks[j] < blocks[best]:
                    best = j

        if best != -1:
            allocation[i] = best
            blocks[best] -= processes[i]

    print("\nProcess\tSize\tBlock")
    for i in range(len(processes)):
        if allocation[i] != -1:
            print(f"P{i+1}\t{processes[i]}\tB{allocation[i]+1}")
        else:
            print(f"P{i+1}\t{processes[i]}\tNot Allocated")

blocks = list(map(int, input("Enter memory block sizes: ").split()))
processes = list(map(int, input("Enter process sizes: ").split()))

best_fit(blocks, processes)
```

### Output:

```
Enter memory block sizes: 100 500 200 300 600
Enter process sizes: 212 417 112 426
```

| Process | Size | Block |
|---------|------|-------|
| P1      | 212  | B4    |
| P2      | 417  | B2    |
| P3      | 112  | B3    |
| P4      | 426  | B5    |

### c. Worst Fit

#### Source Code:

```
def worst_fit(blocks, processes):
    allocation = [-1] * len(processes)

    for i in range(len(processes)):
        worst = -1

        for j in range(len(blocks)):
            if blocks[j] >= processes[i]:
                if worst == -1 or blocks[j] > blocks[worst]:
                    worst = j

        if worst != -1:
            allocation[i] = worst
            blocks[worst] -= processes[i]

    print("\nProcess\tSize\tBlock")
    for i in range(len(processes)):
        if allocation[i] != -1:
            print(f"P{i+1}\t\t{processes[i]}\tB{allocation[i]+1}")
        else:
            print(f"P{i+1}\t\t{processes[i]}\tNot Allocated")

    blocks = list(map(int, input("Enter memory block sizes: ").split()))
    processes = list(map(int, input("Enter process sizes: ").split()))

    worst_fit(blocks, processes)
```

#### Output:

```
Enter memory block sizes: 100 500 200 300 600
Enter process sizes: 212 417 112 426
```

| Process | Size | Block         |
|---------|------|---------------|
| P1      | 212  | B5            |
| P2      | 417  | B2            |
| P3      | 112  | B5            |
| P4      | 426  | Not Allocated |

## Lab-5: Write a program to simulate following page replacement algorithm and find the page fault

### a) FIFO

Source Code:

```
def fifo(pages, frames):
    memory = []
    page_faults = 0
    for page in pages:
        if page not in memory:
            page_faults += 1
            if len(memory) < frames:
                memory.append(page)
            else:
                memory.pop(0)
                memory.append(page)
        print(page, "->", memory)
    print("\nTotal Page Faults =", page_faults)
pages = list(map(int, input("Enter page reference string: ").split()))
frames = int(input("Enter number of frames: "))
fifo(pages, frames)
```

Output:

```
Enter page reference string: 1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3
1 -> [1]
2 -> [1, 2]
3 -> [1, 2, 3]
4 -> [2, 3, 4]
1 -> [3, 4, 1]
2 -> [4, 1, 2]
5 -> [1, 2, 5]
1 -> [1, 2, 5]
2 -> [1, 2, 5]
3 -> [2, 5, 3]
4 -> [5, 3, 4]
5 -> [5, 3, 4]

Total Page Faults = 9
```

### b) LRU

Source Code:

```
def lru(pages, frames):
    memory = []
    page_faults = 0
    for page in pages:
        if page in memory:
            memory.remove(page)
            memory.append(page)
        else:
            page_faults += 1
            if len(memory) >= frames:
                memory.pop(0)
            memory.append(page)
        print(page, "->", memory)
    print("\nTotal Page Faults =", page_faults)
pages = list(map(int, input("Enter page reference string: ").split()))
frames = int(input("Enter number of frames: "))

lru(pages, frames)
```

## Output:

```
Enter page reference string: 1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3
1 -> [1]
2 -> [1, 2]
3 -> [1, 2, 3]
4 -> [2, 3, 4]
1 -> [3, 4, 1]
2 -> [4, 1, 2]
5 -> [1, 2, 5]
1 -> [2, 5, 1]
2 -> [5, 1, 2]
3 -> [1, 2, 3]
4 -> [2, 3, 4]
5 -> [3, 4, 5]

Total Page Faults = 10
```

## c) OPR

### Source Code:

```
def optimal(pages, frames):
    memory = []
    page_faults = 0
    for i in range(len(pages)):
        page = pages[i]
        if page not in memory:
            page_faults += 1
            if len(memory) < frames:
                memory.append(page)
            else:
                future = pages[i + 1:]
                indexes = []
                for p in memory:
                    if p in future:
                        indexes.append(future.index(p))
                    else:
                        indexes.append(float('inf'))
                replace_index = indexes.index(max(indexes))
                memory[replace_index] = page
        print(page, "->", memory)
    print("\nTotal Page Faults =", page_faults)
pages = list(map(int, input("Enter page reference string: ").split()))
frames = int(input("Enter number of frames: "))

optimal(pages, frames)
```

## Output:

```
Enter page reference string: 1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3
1 -> [1]
2 -> [1, 2]
3 -> [1, 2, 3]
4 -> [1, 2, 4]
1 -> [1, 2, 4]
2 -> [1, 2, 4]
5 -> [1, 2, 5]
1 -> [1, 2, 5]
2 -> [1, 2, 5]
3 -> [3, 2, 5]
4 -> [4, 2, 5]
5 -> [4, 2, 5]

Total Page Faults = 7
```



## Lab-6: Write a Program to simulate following disk scheduling algorithms

### a) FCFS

#### Source Code:

```
def fcfs(requests, head):
    movement = 0
    current = head
    print("\nOrder:", current, end="")
    for request in requests:
        movement += abs(request - current)
        current = request
        print(" ->", current, end="")
    print("\nTotal Head Movement =", movement)
requests = list(map(int, input("Enter disk requests: ").split()))
head = int(input("Enter initial head position: "))
fcfs(requests, head)
```

#### Output:

```
Enter disk requests: 98 183 37 122 14 124 65 67
Enter initial head position: 53

Order: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total Head Movement = 640
```

### b) SSTF

#### Source Code:

```
def sstf(requests, head):
    requests = requests.copy()
    movement = 0
    current = head
    print("\nOrder:", current, end="")
    while requests:
        nearest = min(requests, key=lambda x: abs(x - current))
        movement += abs(nearest - current)
        current = nearest
        requests.remove(nearest)
        print(" ->", current, end="")
    print("\nTotal Head Movement =", movement)
requests = list(map(int, input("Enter disk requests: ").split()))
head = int(input("Enter initial head position: "))
sstf(requests, head)
```

#### Output:

```
Enter disk requests: 98 183 37 122 14 124 65 67
Enter initial head position: 53

Order: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total Head Movement = 236
```

## c) SCAN

### Source Code:

```
def scan(requests, head, disk_size):
    left = [x for x in requests if x < head]
    right = [x for x in requests if x >= head]
    left.sort()
    right.sort()
    movement = 0
    current = head
    print("\nOrder:", current, end="")

    # Move towards higher cylinder numbers
    for request in right:
        movement += abs(request - current)
        current = request
        print(" ->", current, end="")

    # Go to the end of the disk
    if current != disk_size - 1:
        movement += abs((disk_size - 1) - current)
        current = disk_size - 1
        print(" ->", current, end="")

    # Reverse direction
    for request in reversed(left):
        movement += abs(request - current)
        current = request
        print(" ->", current, end="")

    print("\nTotal Head Movement =", movement)
requests = list(map(int, input("Enter disk requests: ").split()))
head = int(input("Enter initial head position: "))
disk_size = int(input("Enter disk size: "))
scan(requests, head, disk_size)
```

### Output:

```
Enter disk requests: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size: 200

Order: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 37 -> 14
Total Head Movement = 331
```